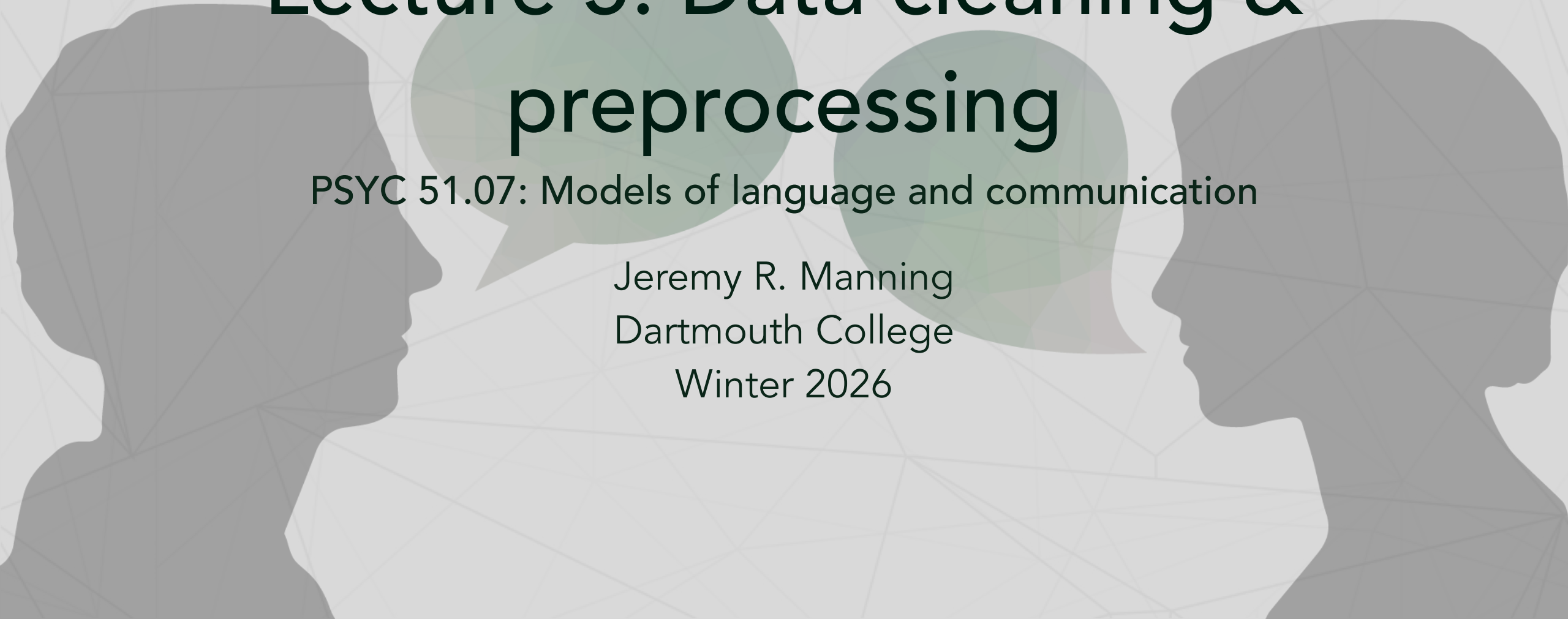
Lecture 5: Data cleaning & preprocessing

PSYC 51.07: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

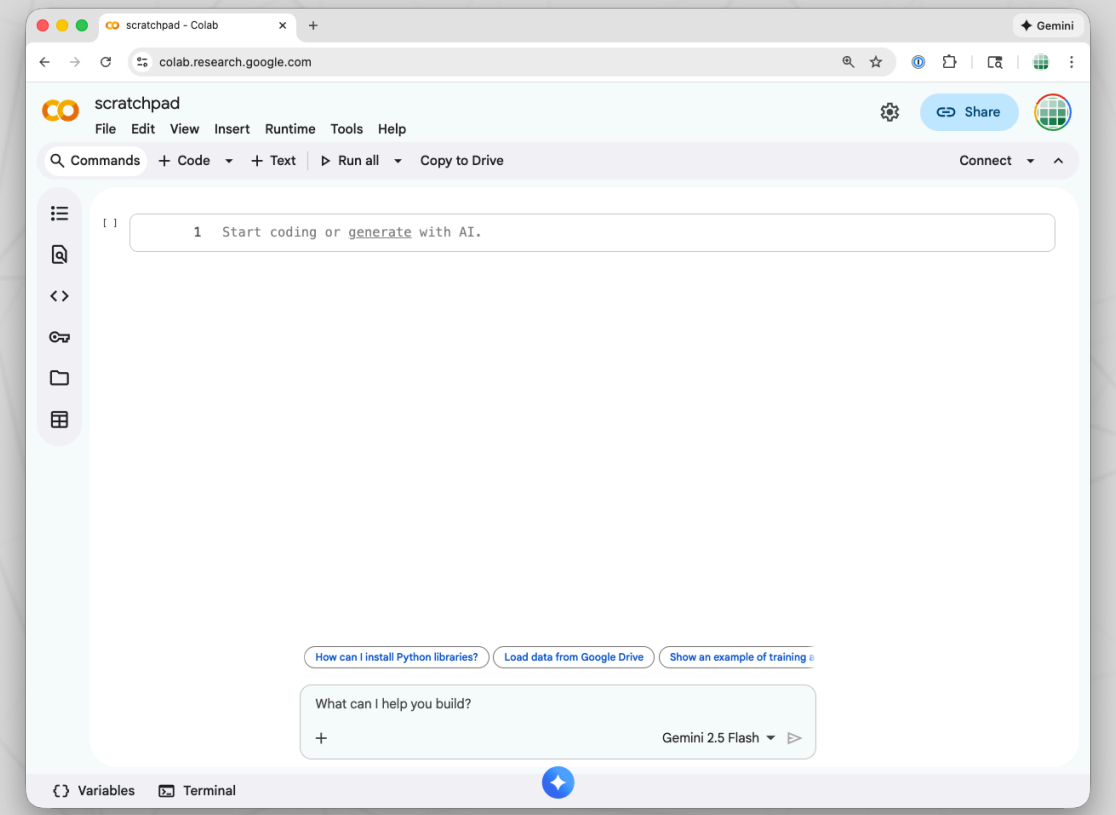
By the end of this lecture, you will be able to...

1. Understand why data cleaning is critical for NLP tasks
2. Apply common preprocessing techniques to raw text
3. Use web scraping to collect text data
4. Implement lemmatization and stemming
5. Make informed decisions about preprocessing strategies

Key theme

Garbage in, garbage out!

Remember: follow along with Google Colab!



- Go to colab.research.google.com
- Click "New notebook"
- Click to create new `text` or `code` cells
- Copy and paste code from slides
- Press `Shift + Enter` to run cells
- *Doing* is the best way to learn!
- **Experiment! Break stuff!**

Real-world text is messy

Common problems:

- HTML/XML tags: `<p>text</p>`
- Special characters: &
- Inconsistent formatting
- Extra whitespace
- Mixed encodings (UTF-8, ASCII...)
- Emoji and Unicode characters

Consequences of dirty data:

- Models learn noise, not signal
- Reduced accuracy and consistency
- Poor generalization to new data
- Wasted computation on junk
- Unpredictable model behavior
- Difficulty debugging issues

Remember

"Quality of input ∝ quality of output"

The data cleaning pipeline

A typical data cleaning pipeline

Important

Pipeline varies by task— not all steps are always needed...and sometimes you'll need *additional* steps!

Rule of thumb

Think about: what are you trying to *get* out of your dataset? If something in your dataset is **informative**, keep it; otherwise normalize or remove it to cut down on how much explanatory power is eaten up by the stuff you don't think matters.

Regular expressions are often useful for cleaning

1. Remove HTML tags
2. Remove URLs/emails
3. Normalize whitespace

Test it!

Check your patterns against example inputs (you can make them up or use a package like Faker). Make sure you've covered weird edge cases in addition to common patterns.

Be careful...

When you normalize, remove, or simplify text, it introduces a potentially tricky-to-debug condition: text that *starts out* different may end up being the *same* after "cleaning."

```
1  import re
2  raw = "<p>I LOVE this!!!
   https://ex.com a@b.com</p>"
3
4  text = re.sub(r'<.*?>', '',
   raw)   # HTML tags
5  text = re.sub(r'http\S+', '',
   text)  # URLs
6  text = re.sub(r'\S+@\S+', '',
   text)  # ← Emails ↓ Whitespace
7  text = ' '.join(text.split())
8
9  print(text) # "I LOVE this!!!"
```

Example: multi-step text cleaning

Stage	Text
Raw	`<p>I LOVE this product 😄!!! https://ex.com</p>`
Remove HTML	`I LOVE this product 😄!!! https://ex.com`
Remove URLs	`I LOVE this product 😄!!!`
Remove emojis	`I LOVE this product !!!`
Normalize spaces	`I LOVE this product!!!`
Lowercase	`i love this product!!!`
Remove extra punctuation	`i love this product!`

Example decision points

- Keep emoji? **Yes** for sentiment analysis (conveys emotion)
- Keep punctuation!!!? **Maybe** (emphasis, but noisy)
- Lowercase? **Depends** on task (lose "LOVE" emphasis)

Different tasks need different preprocessing

Task	Keep	Remove
Sentiment analysis	Emoji, punctuation (style matters)	Extra whitespace, special chars
Named entity recognition	Case (proper nouns important)	Extra whitespace
Topic modeling (we'll learn about this next week!)	Content words only	Punctuation, case (lowercase all)
Text classification	Depends on domain	HTML tags, URLs, special chars
Training LLMs	As much raw text as possible (let model learn patterns)	Broken encodings, invalid chars

Pro tip

When in doubt, keep it! If your dataset is large enough, modern models can often learn to ignore irrelevant noise.

The web is a massive source of text data

What you can collect:

- News articles and blog posts
- Product reviews and ratings
- Social media and forum discussions
- Domain-specific technical content

Popular tools:

- **Beautiful Soup:** Parse HTML/XML
- **Requests:** Fetch web pages
- **Scrapy:** Full scraping framework
- **Selenium:** JavaScript-heavy sites

Scrape ethically!

Honor terms of service and copyright, respect rate limits, and check robots.txt files.

Basic web scraping with Beautiful Soup

```python
from bs4 import BeautifulSoup
import requests

# Fetch webpage
url = "https://example.com/article"
response = requests.get(url)
html_content = response.content

# Parse HTML
soup = BeautifulSoup(html_content, 'html.parser')

# Extract text from specific elements
title = soup.find('h1').get_text()
article = soup.find('article').get_text()
clean_text = ' '.join(article.split())  # Remove extra whitespace
print(f"Title: {title}\nArticle: {clean_text[:200]}... ")
```

Advanced Beautiful Soup techniques

```
from bs4 import BeautifulSoup

html = """<div class="article">
 <h2>Breaking News</h2>
 <p class="content">First paragraph.</p>
 <p class="content">Second paragraph.</p>
 <div class="ads">Advertisement</div>
</div>"""

soup = BeautifulSoup(html, 'html.parser')

paragraphs = soup.find_all('p', class_='content')  # Find content paragraphs
text = ' '.join([p.get_text() for p in paragraphs])

for ad in soup.find_all('div', class_='ads'):      # Remove unwanted sections
    ad.decompose()

article_text = soup.get_text(separator=' ', strip=True)
print(article_text)
```

Normalizing individual words: *stemming* is fast but crude; *lemmatization* is accurate

Stemming

- Rule-based crude chopping
- Fast and simple
- May produce non-words
- No additional context needed
- Example: "running" → "run"

Lemmatization

- Uses vocabulary + morphology
- Slower but more accurate
- Always produces real words
- Requires POS context
- Example: "better" → "good"

When to use which?

- **Stemming:** Speed matters, approximate matching OK
- **Lemmatization:** Need interpretable, real words
- There are several variants of each of these approaches

Lemmatization produces real words; stemming produces fragments

Word	Porter Stemmer	Lemmatizer	Notes
ran	ran	run	Lemma recognizes irregular verb
running	run	run	Both work well
runner	runner	runner	Both keep as-is
better	better	good	Lemma handles irregular adjective
best	best	good	Lemma handles superlative
geese	gees	goose	Stemmer produces non-word!
studies	studi	study	Stemmer produces non-word!
organizing	organ	organize	Stemmer too aggressive!

Stemming with NLTK

```python
from nltk.stem import PorterStemmer, SnowballStemmer
porter = PorterStemmer()
snowball = SnowballStemmer('english')

words = ['running', 'runs', 'runner', 'ran', 'easily', 'fairly']

print("Word          Porter       Snowball")
for word in words:
    print(f"{word:12} {porter.stem(word):12} {snowball.stem(word)}")

# Output:
# running      run          run
# runs         run          run
# runner       runner       runner
# ran          ran          ran
# easily       easili       easili
# fairly       fairli       fair
```

Lemmatization with spaCy

```python
import spacy
nlp = spacy.load("en_core_web_sm")

text = "The cats are running faster than dogs ran yesterday"
doc = nlp(text)

print(f"{'Word':<12} {'Lemma':<12} {'POS'}")
print("-" * 36)
for token in doc:
    print(f"{token.text:<12} {token.lemma_:<12} {token.pos_}")

# Output:
# Word         Lemma        POS
# cats         cat          NOUN
# running      run          VERB
# dogs         dog          NOUN
# ran          run          VERB
```

spaCy vs. NLTK for lemmatization

Feature	spaCy	NLTK
Accuracy	High	Good
Setup	Easy	Requires data download
POS tagging	Built-in	Separate step
Dependencies	Included	Manual WordNet
Use case	Production	Research/Teaching

Recommendation

Use spaCy for most tasks! It's faster, more accurate, and easier to use in production.

Stop words: common words with little semantic value

When to remove:

- Topic modeling*
- Text classification
- Search engines
- Information retrieval

When to keep:

- Sentiment ("not good" ≠ "good")
- Machine translation
- Text generation
- Neural models (they learn!)

Examples

"the", "a", "is", "in", "and", "or" — language-specific lists available in NLTK, spaCy, and scikit-learn (among others).

Topic modeling

*There's that term again! **Topic modeling** is an unsupervised learning technique that identifies topics in a collection of documents by analyzing word co-occurrence patterns (i.e., "groups of words that often appear together"). Stop words are so common that they appear in nearly every document, so they aren't helpful for *distinguishing* topics.

Stop words in practice

```python
from nltk.corpus import stopwords
import spacy

stop_words_nltk = set(stopwords.words('english'))  # NLTK approach
text = "This is an example showing stop word removal"
filtered_nltk = [w for w in text.lower().split() if w not in stop_words_nltk]
print("NLTK:", filtered_nltk)  # ['example', 'showing', 'stop', 'word', 'removal']

nlp = spacy.load("en_core_web_sm")  # spaCy approach
doc = nlp(text)
filtered_spacy = [token.text for token in doc if not token.is_stop]
print("spaCy:", filtered_spacy)  # ['example', 'showing', 'stop', 'word', 'removal']
```

Discussion: preprocessing trade-offs

What do you think?

1. *Information loss:* What do we lose when we lowercase everything?
   - Think: "US" (United States) vs. "us" (pronoun)
2. *Task dependency:* why might we use different preprocessing steps for different tasks?
   - Hint: which signals matter for sentiment vs. topic modeling?
3. *Modern models:* when might we skip preprocessing entirely?
   - Consider: size and quality of source data, model architecture
4. *Bias introduction:* can preprocessing introduce bias?
   - Example: Removing slang, intentional misspellings, or dialectal variations

Can you think of other examples?

What other aspects of text preprocessing do you think might be important to consider? Have you ever encountered a situation where you had to preprocess text data in a specific way to achieve better results? How did you decide on the best approach?

Practical tips for data cleaning

Best practices

1. **Inspect your data first!** — Look at samples before deciding on preprocessing
2. **Keep raw data separate** — Never overwrite originals; you might need them!
3. **Document your pipeline** — Track what preprocessing you applied and why
4. **Experiment!** — Try different approaches, measure impact on task
5. **Validate incrementally** — Check results after each preprocessing step
6. **Consider automation** — Use libraries: spaCy, NLTK, HuggingFace tokenizers

Libraries and resources for text preprocessing

Web scraping

- Beautiful Soup: Parse HTML/XML
- Requests: Fetch web pages
- Scrapy: Full scraping framework
- Selenium: JavaScript-heavy sites

Text processing

- spaCy: Industrial-strength NLP
- NLTK: Natural Language Toolkit
- TextBlob: Simple text processing
- Gensim: Topic modeling and more
- Scikit-learn: ML with text features

Character encoding

- chardet: Character encoding detection
- ftfy: Fixes broken Unicode

HuggingFace Resources

- Chapter 3.2: Processing Data
- Chapter 2.4: Tokenizers
- Tokenizers library

Other useful libraries

- regex: Enhanced regex capabilities
- Faker: Generate fake data for testing

Further reading

Classic papers

- **Porter, M. F. (1980, *Program*).** An algorithm for suffix stripping.
- **Manning, C. D. et al. (2008, *Introduction to information retrieval*).** Text preprocessing fundamentals.

Ethical considerations

Liang et al. (2020, *arXiv*). Towards Debiasing Sentence Representations.

Let's try it out!

Time to get your hands dirty...

1. Choose *any* website (news, blog, Reddit...)
2. Scrape 10-20 articles/posts
3. Apply different preprocessing pipelines:
   - **Minimal:** Just remove HTML
   - **Moderate:** + normalize whitespace, lowercase
   - **Heavy:** + lemmatize, remove stop words
4. Compare the results:
   - How does vocabulary size change?
   - What information is lost/preserved?
   - What'd you get stuck on?

Bonus

Share what you learn with the class, live or on our Discord!

Key takeaways

1. **Preprocessing is crucial** but task-dependent: no universal pipeline
2. **Balance cleaning vs. information loss**: more preprocessing does not always mean better
3. **Stemming vs. lemmatization**: the correct answer is "lemmatization"
4. **Big datasets and complex models** can handle messy text better than older models fit to small datasets
5. **Always validate**: inspect your data before and after preprocessing

What's next?

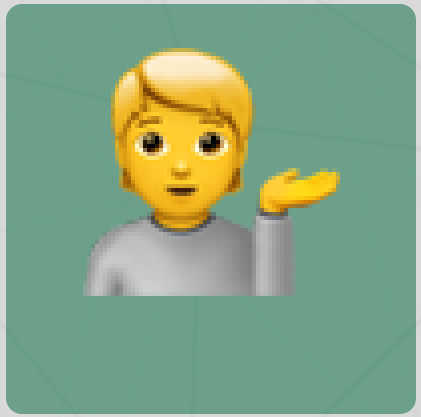
Rest of this week

- **Wednesday**: tokenization— carving text into useful pieces
- **Thursday (X-hour)**: text classification— automatically labeling text data
- **Friday**: POS tagging and sentiment analysis— understanding word roles and emotions

Questions? Want to chat more?

Email me

Join our Discord



Come to office hours

Remember

- Assignment 1 is due on Friday; ask questions early and often!
- Feeling lost or stuck? Let me know; I'm here to help!